UniSENAI

Estruturas de Dados

Aula 02a: Funções/métodos (definição, motivação, parametrização, retorno e escopo); Recursividade (definição, funcionamento, estrutura e exemplos); Iteratividade vs. Recursividade.

Matheus Luan Krueger, M.e

UniSENAI

Roteiro

- Funções/métodos
  - definição
  - motivação
  - assinatura
  - exemplo
  - retorno/parâmetros
  - escopo
- Programação iterativa
- Recursividade
  - definição
  - funcionamento
  - estrutura
  - exemplos
- Iteratividade vs. Recursividade

UniSENAI

~~Funções~~/métodos: definição

- **Conjunto de comandos agrupados em um bloco**, que recebe um "nome" e, através deste, pode ser chamado sempre quando necessário.
- Um método é definido por uma **assinatura**, que, além do seu **nome**, é composta por:
  - **modificadores** – tipo de permissão ao método.
  - **retorno** – tipo de retorno do método ou void caso não tenha retorno.
  - **parâmetros** – atributos utilizados dentro do processamento do método.
  - **exceções** – informação de algum erro que pode ocorrer na execução do método.

```
public static void main(String[] args) {
```

UniSENAI

~~Funções~~/métodos: motivação

- Permitir o reaproveitamento de código já construído (por você ou por outros programadores);
- Evitar que um trecho de código que seja repetido várias vezes dentro de um mesmo programa;
- Permitir a alteração de um trecho de código de uma forma mais rápida. Com o uso de uma função é preciso alterar apenas dentro da função que se deseja;
- Para que os blocos do programa não fiquem grandes demais e, por consequência, mais difíceis de entender;
- Facilitar a leitura do programa-fonte de uma forma mais fácil;
- Para separar o programa em partes (blocos) que possam ser logicamente compreendidos de forma isolada.

```java
public class Programa01 {

    public static void main(String args[]) {

      Scanner leitor = new Scanner(System.in);

      double notas[] = new double[3];

      System.out.print("Informe uma nota: ");
      notas[0] = leitor.nextDouble();
      System.out.print("Informe uma nota: ");
      notas[1] = leitor.nextDouble();
      System.out.print("Informe uma nota: ");
      notas[2] = leitor.nextDouble();
    }
}
```

```java
public class Programa02 {

    public static void main(String args[]) {

      Scanner leitor = new Scanner(System.in);

      double notas[] = new double[3];

      System.out.print("Informe uma nota: ");
      notas[0] = leitor.nextDouble();
      System.out.print("Informe uma nota: ");
      notas[1] = leitor.nextDouble();

      boolean entradaValida = false;
      while (!entradaValida) {
        System.out.print("Informe uma nota: ");
        notas[2] = leitor.nextDouble();
        if (notas[2] < 0 || notas[2] > 10) {
            System.out.println("Nota inválida, informe novamente!");
        }
      }
    }
}
```

UniSENAI

~~Funções~~/métodos: assinatura

```
public static void main (String[] args) [...] {
    System.out.println(“Mensagem 123”);
}
```

UniSENAI

~~Funções~~/métodos: assinatura

modificadores

```
public static void main (String[] args) [...] {
    System.out.println("Mensagem 123");
}
```

Modificadores de acesso:
- private
- protected
- default (package-private)
- public

Outros modificadores:
- abstract
- final
- static
- native
- transient
- synchronized

UniSENAI

~~Funções~~/métodos: assinatura

tipo de retorno

```
public static void main (String[] args) [...] {
    System.out.println("Mensagem 123");
}
```

UniSENAI

~~Funções~~/métodos: assinatura

nome do método

```
public static void main (String[] args) [...] {
    System.out.println("Mensagem 123");
}
```

UniSENAI

~~Funções~~/métodos: assinatura

parâmetros

```
public static void main (String[] args) [...] {
    System.out.println("Mensagem 123");
}
```

UniSENAI

~~Funções~~/métodos: assinatura

notificação de exceções

```
public static void main (String[] args) [...] {
    System.out.println("Mensagem 123");
}
```

UniSENAI

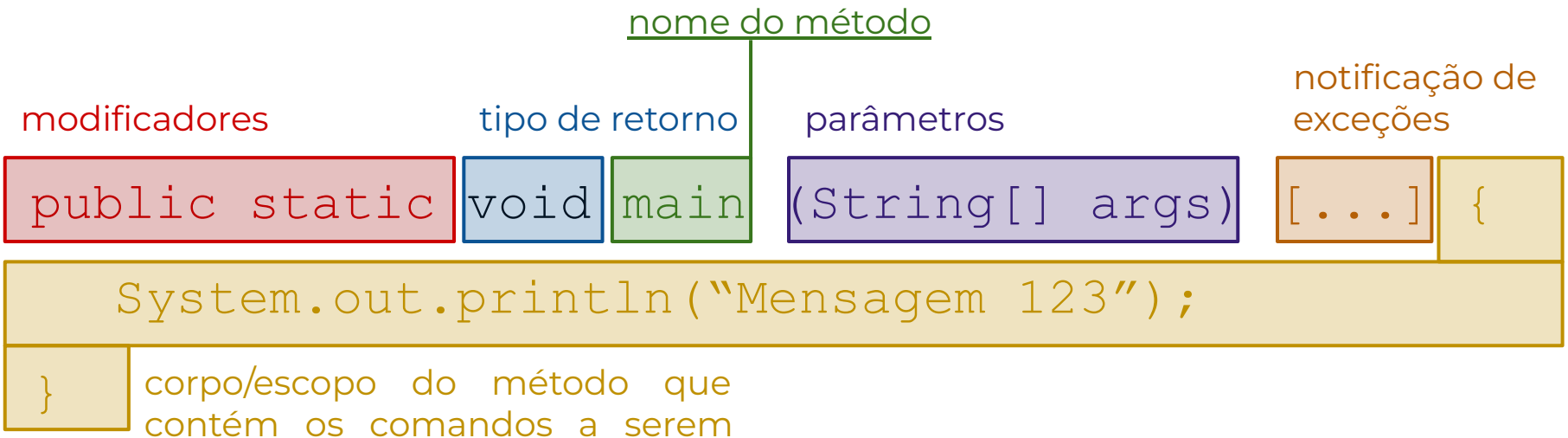
~~Funções~~/métodos: assinatura

```
public static void main (String[] args) [...] {
    System.out.println("Mensagem 123");
}
```

corpo/escopo do método que contém os comandos a serem executados, definido pelo par de chaves { }

UniSENAI

~~Funções~~/métodos: assinatura



UniSENAI

~~Funções~~/métodos: exemplo

- Em primeiro lugar, imaginemos que você necessita várias vezes em seu programa imprimir a mensagem "Passei por aqui...". Uma alternativa é **criar um método**. Com o uso de métodos, este processo fica simplificado, bastando, sempre que necessário, 'chamar/invocar' o método que imprime a mensagem.
- Observe o exemplo a seguir:

```
1  public class TesteMetodo {
2      public static void main(String args[]) {
3          mostrarMensagem(); // 1ª chamada do método
4
5          mostrarMensagem(); // 2ª chamada do método
6
7          mostrarMensagem(); // 3ª chamada do método
8      }
9
10     public static void mostrarMensagem() {
11         System.out.println("Passei por aqui...");
12     }
13 }
```

Desafio:

Como modificar este programa de forma que um **parâmetro inteiro** seja adicionado na assinatura do método e modificar a sua lógica de programação? Dentro da implementação do método, essa nova informação inteira passada por parâmetro deverá ser utilizada para implementar uma estrutura de repetição para apresentar 'n' vezes a mensagem.

UniSENAI

Resolução

- Note que, para resolver o desafio proposto anteriormente, necessitamos modificar o código fonte para que o método **receba um parâmetro inteiro**. Assim, conseguiremos utilizar esse dado para **condicionar a nossa estrutura de repetição**:

```
public static void mostrarMensagem(int quantidade) {
    for (int i=0; i<quantidade; i++) {
        System.out.println("Passei por aqui…");
    }
}
```

UniSENAI

~~Funções~~/métodos: parâmetros

- A linguagem Java permite o uso de **parâmetros**. Este parâmetros possibilitam a troca de mensagens. Para definir os parâmetros de um método **o programador deve explicitá-los como se estive declarando uma variável**, entre os parênteses da assinatura do método.
- Caso precise declarar mais de um parâmetro, basta separá-los por vírgulas. No exemplo a seguir temos o método somaDeInteiros que possui dois parâmetros, sendo ambos do tipo int.

```
1  public class TesteMetodo {
2      public static void main(String args[]) {
3          int x = 4;
4          int y = 3;
5          somaDeInteiros(x, y);
6      }
7
8      public static void somaDeInteiros(int a, int b) {
9          int resultado = a+b;
10         System.out.println("A soma entre "+a+" e "+b+" é a "+resultado);
11     }
12 }
```

Desafio:
Como modificar este programa de forma que um **retorno inteiro** seja adicionado na assinatura do método e modificar a sua lógica de programação? Ao invés de imprimir o resultado da soma, deve-se retorná-la... A impressão acontecerá dentro do método main (local que chamou o seu método somaDeInteiros).

UniSENAI

Resolução

- Note que, para resolver o desafio proposto anteriormente, necessitamos modificar o código fonte para que o método **tenha um retorno inteiro**. Assim, ao invés de imprimirmos o resultado dentro do método, **retornamos o resultado da soma para que o invocou/chamou**.

```
public static int somaDeInteiros(int a, int b) {
    int resultado = a + b;
    return resultado;          // ou diretamente: return (a+b);
}
```

UniSENAI

~~Funções~~/métodos: escopo de variáveis

- Por escopo de uma variável entende-se **o bloco de código onde esta variável é válida**. Com base nisto, temos as seguintes afirmações:
  - variáveis "valem"/existem no bloco que são definidas;
  - variáveis definidas dentro de um método recebem o nome de **variáveis locais**;
  - parâmetros formais de um método valem também somente dentro do método;
  - variável definida dentro de um método não é acessível em outros métodos, MESMO ESTAS VARIÁVEIS TENHAM NOME IDÊNTICOS.

Qual o escopo de x, y?
Qual o escopo de a, b?
Qual o escopo de resultado?
E se tivéssemos um atributo, qual seu escopo? Onde ele seria acessível?

```
1  public class TesteMetodo {
2      public static void main(String args[]) {
3        int x = 4;
4        int y = 3;
5        somaDeInteiros(x, y);
6      }
7
8      public static void somaDeInteiros(int a, int b) {
9          int resultado = a+b;
10         System.out.println("A soma entre "+a+" e "+b+" é a "+resultado);
11     }
12 }
```

UniSENAI

~~Funções~~/métodos: escopo de variáveis

```
1  public class TesteMetodo {
2      public static void main(String args[]) {
3        int b;
4        b = 10;
5        System.out.println("Valor de B: "+b);
6        b = 20;
7        metodo1();
8        System.out.println("Valor de B: "+b);
9        b = 30;
10       metodo2();
11       System.out.println("Valor de B: "+b);
12     }
13
14     public static void metodo1() {
15         int b;
16         b = -100;
17         System.out.println("Valor de B dentro do método1: "+b);
18     }
19
20     public static void metodo2() {
21         int b;
22         b = 200;
23         System.out.println("Valor de B dentro do método2: "+b);
24     }
25 }
```

Quais valores de B serão impressos na execução deste algoritmo? Em qual ordem?

```
Valor de B: 10
Valor de B dentro do método1: -100
Valor de B: 20
Valor de B dentro do método2: 200
Valor de B: 30
```

Nota:

Parâmetros de tipos primitivos ➔ passagem por valor

Parâmetros de tipos Objeto (não primitivo) ➔ passagem por referência

UniSENAI

Estruturas de seleção: uma breve revisão

- Declaração if
- Declaração if-else
- Declaração if-else (operador ternário)
- Declaração if-else-if encadeada
- Declaração if / if-else aninhada
- Múltipla escolha (switch)

UniSENAI

Declaração if

```
if (<expr-lógica>) {
    // código se TRUE
}
```

```
public class Teste1 {
     public static void main(String[] args) {
          int nota = 8;
          if (nota >= 6) {
               System.out.println("Aprovado!");
          }
     }
}
```

UniSENAI

Declaração if-else

```
if (<expr-lógica>) {
    // código se TRUE
} else {
    // código se FALSE
}
```

```
public class Teste2 {
    public static void main(String[] args) {
        int nota = 8;
        if (nota >= 6) {
            System.out.println("Aprovado!");
        } else {
            System.out.println("Reprovado!");
        }
    }
}
```

UniSENAI

Declaração if-else (operador ternário)

```
( expr-lógica ? __ : __ )
```

```
public class Teste3 {

     public static void main(String[] args) {

          int num = 8;

          String msg;

          msg = ( (num%2==0) ? "Par" : "Ímpar" );

          System.out.println(msg);

     }

}
```

UniSENAI

Declaração if-else-if encadeada

```
if (<expr-lógica-1>) {
    // código se TRUE (1)
} else if (<expr-lógica-2>) {
    // código se TRUE (2)
} …
else {
    // código se tudo for F
}
```

```
public class Teste4 {
	public static void main(String[] args) {
		int nota = 8;
		if (nota >= 6) {
			System.out.println("Aprovado!");
		} else if (nota >= 3) {
			System.out.println("Em exame!");
		} else {
			System.out.println("Reprovado!");
		}
	}
}
```

UniSENAI

Declaração if / if-else aninhada

```
if (<expr-lógica>) {
     if (<expr-lógica>) {
          // comandos
     }
}

if (<expr-lógica>) {
     // código se TRUE (2)
} else {
     if (<expr-lógica>) {
          // código se TRUE (2)
     } else {
          // código se tudo for F
     }
}
```

```
public class Teste5 {
      public static void main(String[] args) {
            int nota = 8;
            if (nota >= 6) {
                  System.out.println("Aprovado!");
            } else {
                  if (nota >=3) {
                        System.out.println("Em exame!");
                  } else {
                        System.out.println("Reprovado!");
                  }
            }
      }
}
```

UniSENAI

Declaração múltipla escolha (switch)

```
switch (<var>) {
    case 0:
        comandos;
        [break;]
    case 1:
        comandos;
        [break;]
    case 2:
        comandos;
        [break;]
    default:
        comandos;
}
```

```
public class Teste6 {
     public static void main(String[] args) {
          int num = 8;
          switch (num) {
               case 0:
                    System.out.println("zero");
                    break;
               case 2:
                    System.out.println("dois");
                    break;
               default:
                    System.out.println("outro");
          }
     }
}
```

UniSENAI

Estruturas de repetição: uma breve revisão

• For *(para)*

```
for (<ini>; <expr>; <inc>) {
    //                comandos
}
```

Permite que uma **variável contadora**, seja testada e incrementada a cada iteração, sendo essas informações definidas na chamada do comando.

• While *(enquanto)*

```
while ( <expr-lógica> ) {
    //                comandos
}
```

Determina que, enquanto uma condição for **válida**, o bloco de código será executado. A condição é **testada antes de executar o código**.

• Do-while *(faça enquanto)*

```
do                          {
    //                comandos
} while (expr-lógica);
```

Enquanto uma determinada condição for **válida**, o bloco de código será executado. Testa a condição após executar o cód., **executando pelo menos 1 vez**.

UniSENAI

Hands-on: programando com métodos (iterativos)

Desenvolver os programas solicitados abaixo, considerando a criação de métodos e suas respectivas chamadas dentro do método main. Crie uma classe **MetodosIterativos.java** e desenvolva os métodos abaixo.

1. Crie um método `somarAte(int numero) : int` que recebe um número como argumento e retorna a soma dos números inteiros até o valor desejado. Dentro do método **main**, peça ao usuário para inserir um número e use o método **somarAte** para realizar a soma. Por fim, apresente a soma ao utilizador.

2. Crie um método `fatorial(int numero) : int` que recebe um número inteiro como argumento e retorna o fatorial deste número. Dentro do método **main**, solicite ao usuário para inserir um número e use o método **fatorial** para calcular o fatorial deste número informado. Por fim, apresente o resultado ao utilizador.

3. Crie um método `somarValoresVetor(int valores[]) : int` que recebe um vetor de inteiros como argumento e retorna a soma dos números contidos no vetor. Dentro do método **main**, peça ao usuário um número inteiro para o tamanho do vetor, feito isso, solicite também valores inteiros para popular todas as posições deste vetor. Utilize o método **somarValoresVetor** para realizar a soma. Por fim, apresente a soma ao utilizador.

4. Crie um método `contarCaracteres(char letras[], char letraBuscada) : int` que recebe um vetor de char e uma letra a ser buscada como argumento e retorna a quantidade de vezes que esta letra/char "aparece" nas posições do vetor. Dentro do método **main**, peça ao usuário um número inteiro para o tamanho do vetor, feito isso, solicite também valores char para popular todas as posições deste vetor, bem como uma letra a ser buscada/contabilizada. Utilize o método **contarCaracteres** para realizar esta contabilização. Por fim, apresente a quantidade de vezes que a letra buscada está presente no vetor ao utilizador.

UniSENAI

Recursividade: definição

- **Chamamos de recursividade ou recursão quando uma função chama a si mesma.**
- Recursividade é o **mecanismo de programação** no qual uma definição de função ou de outro objeto refere-se ao próprio objeto sendo definido. Assim, método recursivo é um método que é definido em termos de si mesmo. Recursividade é o mecanismo básico para repetições nas linguagens funcionais.





100 seconds of CS

RECURSION



UniSENAI

Recursividade: funcionamento

- Em um **método recursivo**, a cada chamada é criada na memória uma nova ocorrência do método com comandos e variáveis "isolados" das ocorrências anteriores.
- O método é executado até que todas as ocorrências tenham sido resolvidas.

- **VANTAGENS**
  - Torna a escrita do código **mais simples** e elegante, tornando-o "fácil de entender" (nem sempre essa facilidade é visível) e de manter.
- **DESVANTAGENS**
  - Quando o loop recursivo é muito grande é **consumida muita memória nas chamadas** a diversos níveis de recursão, pois cada chamada recursiva aloca memória para os parâmetros, variáveis locais e de controle.
  - Em muitos casos uma solução iterativa gasta menos memória, e torna-se mais eficiente em termos de performance do que usar recursão.

UniSENAI

Recursividade: estrutura

- Toda recursividade é composta por um **caso base** e pelas **chamadas recursivas**.

  - **Caso base:** é o caso mais simples. É usada uma condição em que se resolve o problema com facilidade.
  - **Chamadas recursivas:** procuram simplificar o problema de tal forma que convergem para o caso base

UniSENAI

Recursividade: exemplo

No programa, se o número n for menor ou igual a 1 o valor 1 será retornado e a função encerrada, sem necessidade de chamadas recursivas. Caso contrário dá-se início a chamadas recursivas até cair no caso mais simples que é resolvido e assim, as chamadas retornam valores de forma a solucionar o cálculo. Veja a figura ao lado que representa as chamadas recursivas para o cálculo de 5!

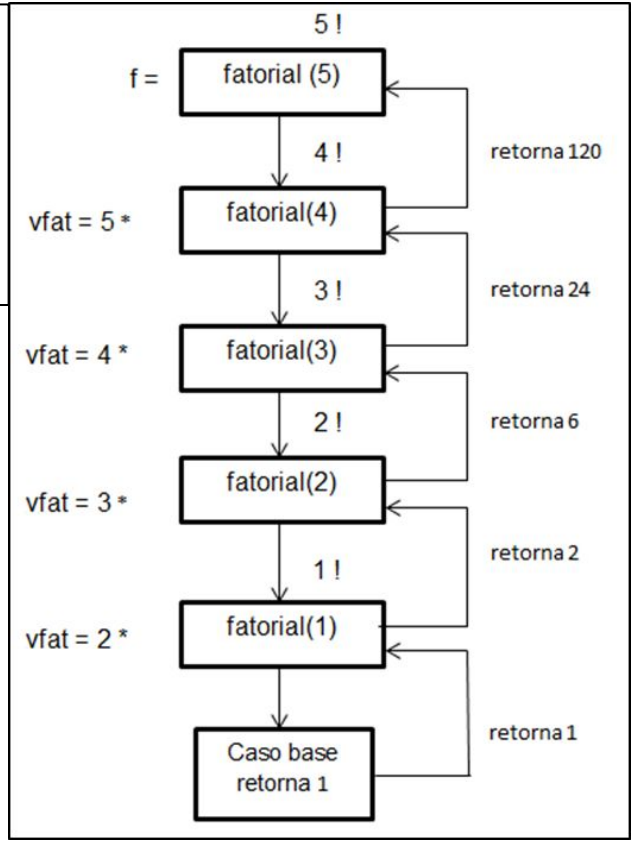
```
public class Teste {
	public static void main(String[] args) {
		int n = 7;
		int fat = fatorial(n);
		System.out.println("O fatorial de " + n + "é" + fat);
	}

	public static int fatorial(int num) {
		int vFat;

		if (num <= 1) {
			return 1;
		} else {
			vFat = num * fatorial(n-1);
			return vFat;
		}
	}
}
```

Caso base

Chamada recursiva



UniSENAI

Recursividade: desafio

- Desenvolva um método recursivo **`potencia(int x, int n) : int`** que, dado dois valores inteiros 'x' e 'n', calcula o valor de x^n.
- O que sabemos sobre potenciação? Conseguimos, a partir de nossos conhecimentos prévios, descobrir/deduzir qual o **caso base** e qual o **passo da recursão**?
  - **Caso base → $x^0 = 1$**
  - **Passo da recursão → $x^n = x * x^{(n-1)}$**
- Como programar?
- Como ficam as chamadas recursivas?

```
public static int potencia (int x, int n) {

      if (n==0) {
            return 1;
      } else {
            return x*potencia(x,(n-1));
      }
}
```

UniSENAI

Hands-on: programando com métodos (recursivos)

Desenvolver os programas solicitados abaixo, considerando a criação de métodos e suas respectivas chamadas dentro do método main. Crie uma classe **MetodosRecursivos.java** e desenvolva os métodos abaixo.

1. Crie um método `somarAte(int numero) : int` que recebe um número como argumento e retorna a soma dos números inteiros até o valor desejado. Dentro do método **main**, peça ao usuário para inserir um número e use o método **somarAte** para realizar a soma. Por fim, apresente a soma ao utilizador.

2. Crie um método `fatorial(int numero) : int` que recebe um número inteiro como argumento e retorna o fatorial deste número. Dentro do método **main**, solicite ao usuário para inserir um número e use o método **fatorial** para calcular o fatorial deste número informado. Por fim, apresente o resultado ao utilizador.

3. Crie um método `somarValoresVetor(int valores[]) : int` que recebe um vetor de inteiros como argumento e retorna a soma dos números contidos no vetor. Dentro do método **main**, peça ao usuário um número inteiro para o tamanho do vetor, feito isso, solicite também valores inteiros para popular todas as posições deste vetor. Utilize o método **somarValoresVetor** para realizar a soma. Por fim, apresente a soma ao utilizador.

4. Crie um método `contarCaracteres(char letras[], char letraBuscada) : int` que recebe um vetor de char e uma letra a ser buscada como argumento e retorna a quantidade de vezes que esta letra/char "aparece" nas posições do vetor. Dentro do método **main**, peça ao usuário um número inteiro para o tamanho do vetor, feito isso, solicite também valores char para popular todas as posições deste vetor, bem como uma letra a ser buscada/contabilizada. Utilize o método **contarCaracteres** para realizar esta contabilização. Por fim, apresente a quantidade de vezes que a letra buscada está presente no vetor ao utilizador.

UniSENAI

Hands-on: resolvendo problemas com recursividade

1. Desenvolver um programa que imprima, de duas formas diferentes, os números de 0 até um determinado valor 'n'. O valor de 'n' deve ser solicitado dentro do método **main**, por meio da interação com o utilizador/usuário. O programa deve conter dois métodos adicionais além do main: Um método iterativo (`imprimirIterativo(int n)`) Um método recursivo (`imprimirRecursivo(int n)`) Ambos os métodos devem receber o valor de 'n' como parâmetro e, quando invocados, devem imprimir na tela todos os números de 0 até 'n', em uma única linha, separados por espaço. Exemplo: ao informar o valor 5, o programa deve imprimir: 0 1 2 3 4 5

2. Desenvolver um programa para calcular, por meio de um método recursivo (`somatorio(int x) : int`), o somatório de todos os números de 1 até um determinado valor 'x'. O valor de 'x' deve ser solicitado dentro do método **main**, a partir da interação com o utilizador/usuário. Uma vez que o somatório fora calculado, deve-se apresentar o resultado na tela. Exemplo: ao informar 10, o programa deve mostrar 55, isso porque 1+2+3+4+5+6+7+8+9+10 = 5.

3. Desenvolver um programa para calcular, por meio de um método recursivo (`fibonacci(int n) : int`), o n-ésimo termo da sequência de Fibonacci. No método main, peça ao usuário para inserir um número e chame o método **fibonacci** para calcular o termo correspondente. Exemplo: ao informar 4, o programa deve mostrar o valor 5, isso porque 5 é o 4º termo da sequência de Fibonacci. Lembre-se que a sequência de Fibonacci é uma sequência de números em que cada número subsequente é a soma dos dois anteriores. A sequência geralmente começa com 0 e 1. Portanto, os primeiros termos da sequência de Fibonacci são: **0, 1, 1, 2, 3, 5, 8, 13, 21, ...** Para encontrar o quarto termo da sequência de Fibonacci, podemos aplicar a definição da sequência. Começamos com os dois primeiros termos: 0 e 1, e então somamos esses dois termos para obter o terceiro termo. Em seguida, somamos o segundo e o terceiro termos para obter o quarto termo:
    i. 0 + 1 = 1 (primeiro termo)
    ii. 1 + 1 = 2 (segundo termo)
    iii. 1 + 2 = 3 (terceiro termo)
    iv. 2 + 3 = 5 (quarto termo)

UniSENAI

Oportunidade - English Online 2026 - British Council

- **Que tal começar o ano batendo a sua meta do ano de estudar Inglês?** 🗺️📚
- Se você está procurando fortalecer seu inglês e melhorar suas perspectivas de empregabilidade, o Banco Santander e o British Council estão oferecendo 10.000 vagas para que você possa fazer isso de casa e no seu próprio ritmo, durante 8 semanas. Além disso, se você for um dos 1.500 alunos com melhor desempenho, receberá 20 semanas adicionais de formação com aulas on-line com um professor. Inscreva-se agora! Ah! Você não precisa ter um diploma universitário nem ser cliente do Banco Santander.
- https://app.santanderopenacademy.com/pt-BR/program/english-online-2026-2nd-call

Terá algum custo? –

Sem nenhum custo. Os cursos são financiados pelo Banco Santander. Se desejar, ao final do curso, você poderá se inscrever por conta própria para o próximo nível, mas essa opção é totalmente pessoal e você não terá nenhuma pressão para isso.



UniSENAI

Dúvidas? 😱

- **AVA** (https://ava.sesisenai.org.br/login/index.php)
- **E-mail** (matheus.krueger@edu.sc.senai.br)
- **WhatsApp** (https://wa.me/554733219647)

- **Monitoria**
  - Sábados, das 8h às 12h, na sala HUB.

"A persistência é o caminho do êxito."
Charles Chaplin